

AAD: "La Lógica de Tron en un LLM"

1. Introducción

En esta práctica integrarás un Modelo de Lenguaje Grande (LLM) como motor de decisión para controlar a un agente en un juego adversarial basado en cuadrícula (similar a Tron). Tu agente LLM competirá contra un agente clásico determinista basado en el algoritmo Minimax. El objetivo es sobrevivir, esquivar los rastros y muros, y llegar a la columna de meta opuesta antes que tu rival. El objetivo es dotar a nuestra "Moto de Luz" de sentido común, permitiéndole detectar trampas, evitar callejones sin salida y encerrar al enemigo mediante llamadas a un LLM con las instrucciones y contexto de la partida actual.

2. Configuración de la API de Groq y código aportado

Para dotar de "cerebro" a tu agente usaremos Groq, una plataforma que aloja modelos como Llama 3 de forma gratuita y a altísima velocidad.

Pasos para obtener tu API Key:

1. Entra en <https://console.groq.com/keys>
2. Regístrate o inicia sesión (puedes usar tu cuenta de Google o GitHub).
3. Haz clic en "Create API Key", ponle un nombre y cópiala.
4. Pega esta clave en la variable API_KEY de tu clase AgenteGroq en Java.

El código a utilizar será el de tu práctica de Minimax, al que se le añadirá un AgenteGrok, que exceptuando el *prompt* y la *apikey* se aporta en Moodle:

```
public class AAD {
    public static void main(String[] args) throws InterruptedException {
        String mapaTexto = "#####\n" +
            "#E      #\n" +
            "#      #\n" +
            "#      #\n" +
            "#      #\n" +
            "#      #\n" +
            "#      S #\n" +
            "#####";

        Entorno juego = new Entorno(mapaTexto);

        BusquedaMinimax agenteA = new BusquedaMinimax();
        AgenteGroq agenteB = new AgenteGroq();
        //AgenteInteractivo agenteB = new AgenteInteractivo();

        System.out.println("--- Inicio de la Simulación ---");
        boolean juegoActivo = true;
        int ciclos = 0;

        .....
        class AgenteGroq {
            private static final String API_KEY = "PON_TU_CLAVE_AQUI";
            private static final String MODEL_NAME = "llama-3.1-8b-instant";

            // ... (Código de conexión HTTP) ...

            private String buildEstrategiaBasica(int f, int c, int metaCol, List<String> validas, Entorno mapa) {
                StringBuilder sb = new StringBuilder();
                sb.append("Estás en un laberinto estilo Tron. Tu posición actual es fila ").append(f).append(", columna ")
                    .append(c).append(".\n");
                sb.append("Tu objetivo es cruzar el mapa y llegar a cualquier casilla que esté en la columna ").append(metaCol)
                    .append(".\n");
                sb.append("Los movimientos válidos que puedes hacer sin chocar ahora mismo son: ")
                    .append(String.join(", ", validas)).append(".\n");
                sb.append("Analiza qué movimiento te acerca más a la columna ").append(metaCol)
                    .append(". Explica brevemente tu razonamiento (1-2 líneas). Luego, OBLIGATORIAMENTE, en la última línea escribe: ACCION: X\n");
                // (donde X es N, S, E u O.);
                return sb.toString();
            }
        }
    }
}
```

3. Ingeniería de Prompts (Prompt Engineering)

El prompt básico anterior hará que el modelo se mueva a ciegas o de forma aleatoria, y será aplastado por el Minimax. Tu tarea principal es mejorar este prompt (contexto) para que el LLM entienda realmente cómo jugar. Pistas y reflexiones para diseñar tu prompt:

- ¿Deberías enviarle todo el mapa visual en texto (con # y espacios) o es mejor describirle matemáticamente su entorno (ej. "tienes un muro al Norte")?
- Histórico: Los LLMs no tienen memoria de llamadas anteriores por defecto en esta implementación. ¿Podrías añadirle las últimas 3 posiciones en las que estuvo para que sepa de dónde viene y evite encerrarse?
- Metas: El Minimax calcula distancias exactas. ¿Cómo puedes hacer que el LLM sepa qué significa acercarse a la "columna meta"? Quizá indicándole explícitamente a cuántos pasos está o pidiéndole que razone la distancia antes de responder.
- Cadena de pensamiento (Chain of Thought): Pídele al LLM que analice primero la situación del rival antes de darte su decisión final. Ejemplo: "Razona primero si el enemigo te va a cortar el paso, y luego dime la ACCIÓN".
- Muestra por pantalla antes de tomar la decisión porqué ha tomado esa decisión

Ejemplo de una ejecución

```

--- Ciclo 0 ---
#####
#A          #
#           #
#           #
#           #
#  #       B #
#####
[Agente A] Decide mover: S
[Agente Groq] Observando el entorno y conectando con la API...
[Agente Groq] Razonamiento:
El enemigo está en la posición (Fila:2, Col:1) y su meta está en la columna 13, por lo que
está a 12 columnas de su meta. Mi meta está en la columna 1, por lo que estoy a 12
columnas de la mía. Sin embargo, dado que el enemigo está más cerca de su meta que yo de
la mía, debo intentar bloquearlo moviéndome en la dirección que lo impida.

Mi posición actual es (Fila:5, Col:13) y el enemigo está en la posición (Fila:2, Col:1),
por lo que si me muevo hacia la izquierda (O), puedo bloquear su camino hacia la derecha
(E).

ACCION: O
[Agente Groq] Esperando 2.5s (Rate Limit)...

[Agente B] Decide mover: O

--- Ciclo 1 ---
#####
#a          #
#A          #
#           #
#           #
#  #       Bb #
#####
[Agente A] Decide mover: E
[Agente Groq] Observando el entorno y conectando con la API...
[Agente Groq] Razonamiento:
El enemigo está en la posición (Fila:2, Col:2) y su meta está en la columna 13, por lo que
está a 11 columnas de su meta. Mi meta está en la columna 1, por lo que estoy a 12
columnas de la mía. El enemigo está más cerca de su meta que yo de la mía, por lo que debo
intentar bloquearlo.

Mi posición actual es (Fila:5, Col:12) y puedo moverme hacia la izquierda (O) o hacia
arriba (N). Como el enemigo está en la columna 2, si me muevo hacia la izquierda, podría
bloquear su camino hacia la derecha.

ACCION: O
[Agente Groq] Esperando 2.5s (Rate Limit)...
[Agente B] Decide mover: O

```

4. Plan de Contingencia: Agente Interactivo (Chatbot Manual)

Las APIs gratuitas a veces tienen límites de peticiones (Rate Limits) o pueden caerse. Si no logras que la conexión automática funcione existe un Plan B:

Crea un "AgenteInteractivo". En lugar de llamar a internet por código, deberás copiar el prompt que has diseñado. T, pegarlo en la web de ChatGPT (Gemini, Groq, el que prefieras), leer lo que la IA te recomienda, y luego escribir en la consola de Java (usando la clase Scanner) el comentario que queremos que quede en la partida y la letra de la acción elegida.

Ejemplo de Partida en Modo Interactivo (Chat Manual)

```
Turno 1 (Ciclo 0)

□□ Alumno (Copiando desde la consola de Java al Chatbot):

Estás en la fila 5 y columna 13. Puedes moverte hacia: N, S, O. Dime solo la letra (N, S, E, O) hacia donde debo moverme.

□ Chatbot (ChatGPT / Gemini):

Dado que mi objetivo suele ser cruzar el mapa y estoy en la columna 13, me moveré hacia el oeste para avanzar. O

(El alumno lee esto, ve la letra "O", vuelve a su consola de Java, escribe la tecla O y pulsa Enter. El juego avanza y el enemigo Minimax hace su movimiento).

Turno 2 (Ciclo 1)

□□ Alumno (Copiando el nuevo estado generado por Java):

Estás en la fila 5 y columna 12. Puedes moverte hacia: N, S, O. Dime solo la letra (N, S, E, O) hacia donde debo moverme.

□ Chatbot (ChatGPT / Gemini):

Sigo avanzando. O

(El alumno vuelve a escribir O en la consola de Java. El juego avanza).

Turno 3 (Ciclo 2)

□□ Alumno (Copiando el nuevo estado generado por Java):

Estás en la fila 5 y columna 11. Puedes moverte hacia: N, S. Al O y al E tienes obstáculos. Dime solo la letra (N, S, E, O) hacia donde debo moverme.

□ Chatbot (ChatGPT / Gemini):

Como no puedo ir al Oeste ni al Este, elegiré el Norte para esquivar el obstáculo. N

(El alumno introduce N en la consola...)
```

Nota: Si se usa el mismo hilo de conversación, el LLM recordará los turnos anteriores de forma natural y tomará mejores decisiones. Este método interactivo es la mejor forma de saber si tu prompt funciona antes de pasarlo al código Java.

5. Evaluación y Trabajo a Entregar

1. **Compíte contra el Minimax:** Haz que AgenteGroq con el prompt mejorado juegue contra BusquedaMinimax.
2. **Informe:** Redacta un documento con la evolución de tus prompts. ¿Qué funcionó y qué no?
- Alucinaciones vs Legalidad:** Minimax jamás hace un movimiento ilegal. ¿El LLM intentó moverse contra una pared a pesar de que el prompt decía que no lo hiciera?
3. **Foro de Prácticas (Obligatorio):** Sube la transcripción o captura las mejores partidas que cada versión de un agente LLM haya jugado contra el Minimax para compartirla con tus compañeros. La primera línea del post debe ser un nick name de la estrategia del agente (por ejemplo: "esquiva y

avance”) con un resumen de la partida: distancia a la que llega de la meta y número de ciclos jugados.

Puedes subir las mejoras que quieras hasta que realices la entrega. Describir que añaden las nuevas versiones a las anteriores: (esquiva a una distancia conocida, conoce su historia,...).

Es una competición... pero el único premio será que se sepa si tu agente dura más contra el minimax o incluso gana y la elegancia de tu estrategia.

Rúbrica de Evaluación:

1. Suspense (0 - 4.9) El **código no compila**, la conexión a la API falla sin manejo de errores, o el agente devuelve acciones nulas/ilegales constantemente que rompen la simulación en el primer turno. Se limita a utilizar **el prompt básico** proporcionado en el enunciado sin aportar nada nuevo, o las modificaciones introducidas son irrelevantes e incomprensibles para el modelo. **No se entrega informe, no se sube partida al foro**, o el documento carece totalmente de un análisis de las decisiones tomadas.

2. Aprobado (5.0 – 6.5)

El agente es funcional. Se conecta correctamente a la API (o usa un agente interactivo manual), y la lógica de parseo extrae una acción legal casi siempre. El **prompt incluye una mejora contextual**. El agente LLM sobrevive algunos turnos antes de perder o estrellarse. Se sube al menos la **transcripción de una partida al Foro**. El informe incluye comentarios sobre varias alternativas de prompts que se intentaron y qué problemas básicos se observaron.

3. Bien-Notable (6.5 - 8.9)

El estudiante demuestra comprensión profunda, experimenta con diferentes técnicas y su informe muestra espíritu crítico. Introduce técnicas medias-avanzadas. El informe incluye **comparativas explícitas entre iteraciones de prompts** (ej. "Con el Prompt A el modelo alucinaba paredes, así que en el Prompt B le pasé un listado matemático explícito"). El agente logra **ocasionalmente un rendimiento decente**, esquivando hábilmente obstáculos o acercándose bastante a la meta antes de caer ante el Minimax.

4. Sobresaliente (9.0 - 10)

El trabajo destaca por su excelencia analítica, creatividad en el diseño de prompts y resultados buenos frente a algoritmos deterministas. **(Pre-procesamiento Táctico)**: El estudiante no solo le pasa el mapa "en crudo" al LLM, sino **que programa lógica en Java para extraer variables clave** (heurísticas, distancias relativas, estado de encerramiento) y se las inyecta en el texto del prompt para potenciar al LLM. Uso de Few-Shot Prompting (**le da ejemplos en el propio texto de cómo jugar**) o definición de que es jugar bien y define estrategias que el prompt aplica. **El LLM rara vez o nunca alucina movimientos ilegales. ¡El agente LLM es competitivo!** Logra jugar partidas largas sin cometer errores tontos e incluso, en algunas circunstancias, consigue arrinconar o vencer al Minimax.

Entrega:

Esta actividad académica dirigida es opcional y aporta un 10% de la calificación final. Se entregará antes del examen de teoría de primera convocatoria en un link. En el examen de teoría puede haber un pequeño apartado para responder a alguna cuestión sobre la estrategia implementada para aquellos alumnos que la entreguen. Se trata de una actividad con espíritu de innovación y exploración, no se requerirá un conocimiento teórico sino detalles que aportan una prueba de autoría.

--